

Construindo a Web API da MecâniQA – Fase 2

Autores: Anthony Kevin Missias
Guilherme Brandão de Araújo
Icaro Menezes dos Santos Vinhas
Igor Santos de Britto
João Vitor Souza Bomfim

Agenda

O objetivo desta apresentação é demonstrar o conteúdo do que foi desenvolvido na criação da Web API da Mecânica

1. Contextualização: O problema da Mecânica e o objetivo atual.

2. Conceitos Básicos de OO: Múltiplos construtores e o uso da palavra `this`.

3. Relacionamentos e Multiplicidades: Como mapeamos associações no código e no diagrama de classes (setas e diamantes).

4. Entidade Associativa: O elo entre Pedidos e Peças.

5. Padrões de Projeto e Arquitetura: Implementação do Builder, o papel dos DTOs e as classes utilitárias Mapper.

- **O Desafio:** A MecâniQA precisa avançar além dos cadastros básicos e gerir a lógica complexa de negócios: **Ordens de Serviço (OS)** e **Pedidos de Peças**.



- **O Objetivo:** Modelar relacionamentos avançados no Diagrama de Classes e proteger o domínio da aplicação, introduzindo padrões de projeto estruturais e criacionais, mantendo a persistência em memória.

- **Múltiplos Construtores (Sobrecarga):** Utilizamos construtores vazios (padrão) para facilitar a instanciação por frameworks/Mappers e construtores parametrizados para garantir a criação de objetos com estado inicial válido.
- **A palavra reservada `this`:** É uma referência à própria instância da classe. Usamos para diferenciar os atributos da classe dos parâmetros recebidos no construtor ou método, evitando ambiguidade.

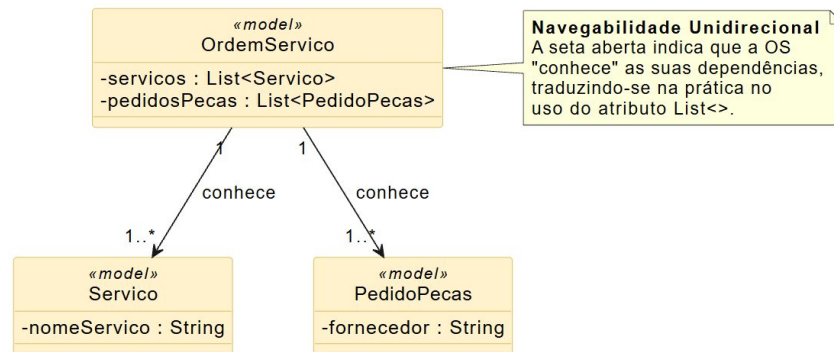
```
public PedidoPecasDTO() {  
    this.itens = new ArrayList<>(); // Construtor 1: Inicializa lista vazia  
}  
  
public PedidoPecasDTO(Long codigoUnico, String fornecedor)  
{  
    this.codigoUnico = codigoUnico; // 'this' diferencia o atributo do parâmetro  
    this.fornecedor = fornecedor; // Construtor 2: Parametrizado  
}
```

Relacionamentos e Multiplicidades

- **Multiplicidades (1..*):** Definem quantos objetos podem participar de uma relação.
- **Mapeamento via Atributos:** Na prática, um relacionamento de "1 para muitos" é mapeado declarando o atributo como uma List<>.
- **Unidirecional vs Bidirecional:** Nossos relacionamentos são navegáveis na direção em que a dependência existe (ex: A OS "conhece" seus serviços através da lista, relacionamento direto).

// O atributo List mapeia a multiplicidade de "1 para muitos" (1..*)

```
private List<Servico> servicos;  
private List<PedidoPecas> pedidosPecas;
```

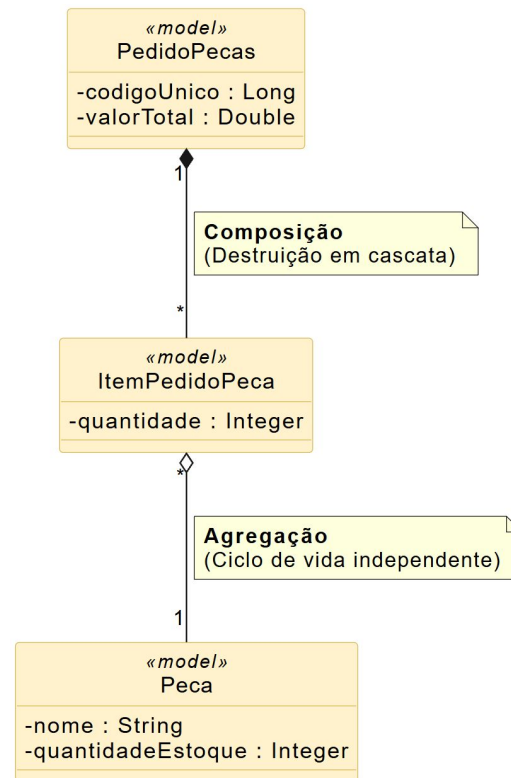


Conceitos do Diagrama de Classes

Setas de Navegabilidade: Linhas com setas abertas indicam relação unidirecional (quem conhece quem).

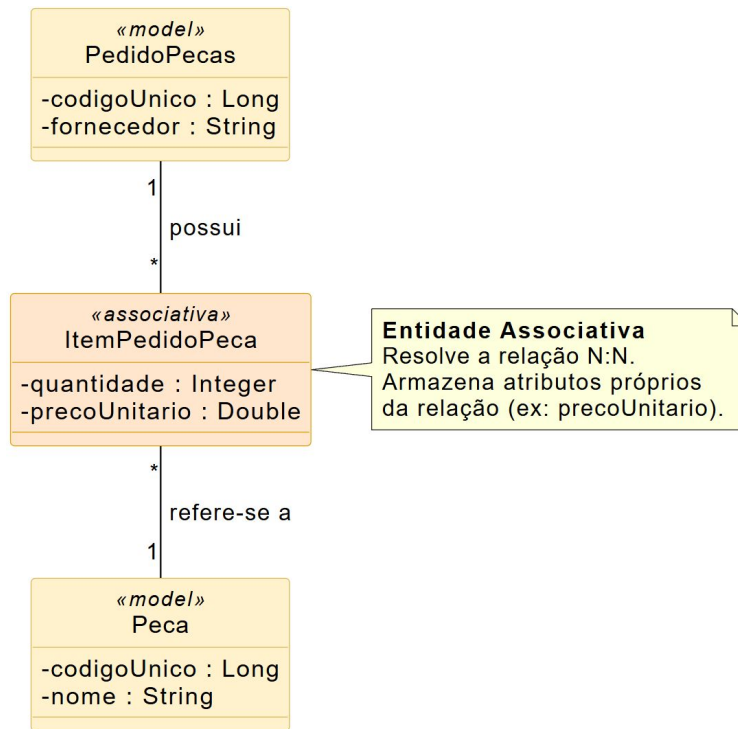
Diamantes (Agregação vs Composição):

- **Agregação** (*Diamante Vazado*): Relação "todo-parte" fraca. Se o Pedido for destruído, a Peça continua existindo no estoque (ciclo de vida independente).
- **Composição** (*Diamante Cheio*): Relação forte. Se o Pedido for destruído, os "Itens do Pedido" também são destruídos (ciclo de vida dependente).
- **Dependência:** Representada por linha tracejada, indica que uma classe (como o Mapper) usa outra temporariamente.



A Entidade Associativa

- **Definição:** Uma Entidade Associativa nasce para resolver relacionamentos NxN, convertendo-os em dois relacionamentos 1xN.
- **Como funciona na MecâniQA:** Um Pedido tem muitas Peças, e uma Peça pode estar em muitos Pedidos. Criamos a classe ItemPedidoPeca.
- **Vantagem:** Permite adicionar atributos próprios à relação, como a quantidade de peças solicitadas naquele pedido específico e o precoUnitario no momento da compra.



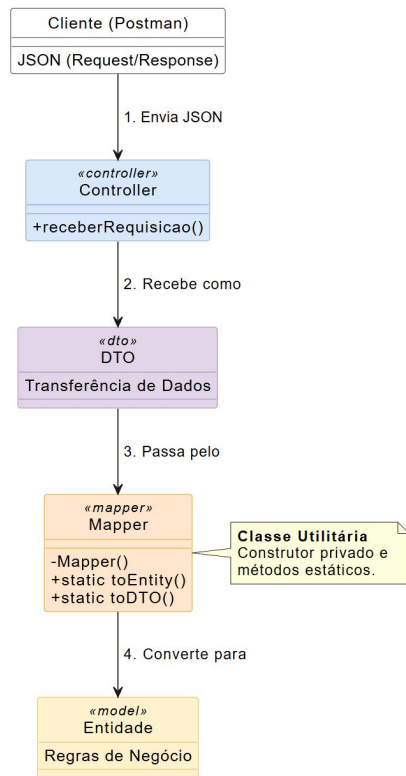
- **Definição Teórica:** Separa a construção de um objeto complexo da sua representação, permitindo criar diferentes representações com o mesmo processo.
- **Objetivo de uso:** Evitar construtores gigantescos (telescópicos) e facilitar a leitura e manutenção.
- **Implementação:** Criamos uma classe interna estática Builder em OrdemServico, com métodos que retornam a própria instância (return this;) para encadeamento, fechando com o método build().

// O Builder em ação no projeto

```
return OrdemServico.builder()  
    .codigoUnico(dto.getCodigoUnico())  
    .descricaoProblema(dto.getDescricaoProblema())  
    .status(dto.getStatus())  
    .valorTotal(dto.getValorTotal())  
    .dataAbertura(dto.getDataAbertura())  
    .servicos(servicos)  
    .pedidosPecas(pedidos)  
    .build();
```

Padrões de Projeto: DTO e Mapper

- **DTO (Data Transfer Object):** Objeto simples usado exclusivamente para transportar dados entre o cliente (API) e o servidor. Evita expor dados sensíveis do modelo (como regras de negócio ou senhas) e reduz chamadas de rede.
- **Mapper (Classe Utilitária):** Responsável por traduzir os dados. Converte o DTO que chegou na requisição em uma Entidade real que o sistema entende (e vice-versa). Possui apenas métodos estáticos e construtor privado, pois não precisa guardar estado (Utility Class).



```
public class OrdemServiceMapper {  
  
    // Construtor privado: impede  
    // instanciação (Classe Utilitária)  
    private OrdemServiceMapper() {}  
  
    // Método estático: acessado  
    // diretamente pela classe  
    public static OrdemService  
    toEntity(OrdemServiceDTO dto) {  
        // Lógica de conversão...  
    }  
}
```

- Pontos de interesse
 - O desenvolvimento da Fase 2 consolidou a nossa compreensão sobre como traduzir regras complexas do mundo real para o Diagrama de Classes e, conseqüentemente, para o código Java.
 - O uso de múltiplos construtores, entidades associativas e o padrão Builder trouxe escalabilidade e segurança para a arquitetura da MecânicaQA.

- Referências do projeto
 - Conteúdos e Materiais Didáticos:
Conteúdos apresentados nas aulas teóricas e Sessões Tutoriais (ST) da Unex.
 - APIs e Controllers:
SPRING BOOT. Spring Boot Reference Documentation. 2026. Disponível em:
<https://docs.spring.io/spring-boot/documentation.html>